

EduGenie

Complete Project Coding

Google Gemini Powered Learning Assistant

ai_client.py

```
"""Shared Gemini client. Set GEMINI_API_KEY in .env to enable cloud features."""
import os
from functools import lru_cache
from google import genai

@lru_cache(maxsize=1)
def get_client():
    key = os.getenv("GEMINI_API_KEY", "").strip()
    if not key:
        raise RuntimeError("GEMINI_API_KEY is missing. Add it to the .env file and restart EduGenie.")
    return genai.Client(api_key=key)

async def generate_text(prompt: str) -> str:
    client = get_client()
    model = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
    # The SDK's synchronous call is used in a worker thread by FastAPI's async endpoint.
    import asyncio
    response = await asyncio.to_thread(
        client.models.generate_content,
        model=model,
        contents=prompt,
    )
    text = getattr(response, "text", None)
    if not text:
        raise RuntimeError("Gemini returned an empty response. Please try again.")
    return text.strip()
```

README.md

EduGenie — Google Gemini Powered Learning Assistant

EduGenie is a FastAPI + HTML/CSS learning assistant based on the supplied project document. It supports:

- Question answering (`/qa`)
- Simple topic explanations (`/explain`)
- Three MCQs with four options each (`/quiz`)
- Passage summarization (`/summarize`)
- Beginner-to-advanced learning paths (`/learn/recommendations`)

Requirements

- Python 3.10 or newer
- A Gemini API key for cloud features
- VS Code (recommended)

The supplied document names Gemini 1.5 Pro. Model availability changes over time, so this implementation defaults to `gemini-2.5-flash`; override

Setup in VS Code (Windows PowerShell)

1. Extract the project ZIP and open the `EduGenie` folder in VS Code.
2. Open **Terminal** → **New Terminal**.
3. Create a virtual environment:

```
```powershell
py -3.10 -m venv .venv
.\.venv\Scripts\Activate.ps1
```
```

If `py` is unavailable, use `python -m venv .venv`.
4. Install dependencies:

```
```powershell
python -m pip install --upgrade pip
pip install -r requirements.txt
```
```
5. Copy `.env.example` to `.env` and add your Gemini API key:

```
```powershell
Copy-Item .env.example .env
```
```

Open `.env` and replace the placeholder. Never share or commit this key.
6. Start the app:

```
```powershell
uvicorn main:app --reload
```
```
7. Open `http://127.0.0.1:8000` in your browser.

macOS / Linux

```
```bash
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
cp .env.example .env
Edit .env and set GEMINI_API_KEY
uvicorn main:app --reload
```
```

Test

- Visit `http://127.0.0.1:8000/health`; expected response: `{"status":"ok"}`.
- In the web page, select each task and submit a short prompt.
- API docs are available at `http://127.0.0.1:8000/docs`.

Example API test:

```
```bash
curl -X POST http://127.0.0.1:8000/qa \
 -H "Content-Type: application/json" \
 -d '{"text":"'Which is the largest ocean?',\"level\":\"beginner\"}'
```
```

Optional local explanation model

To use LaMini for `/explain`, set `USE\_LOCAL\_EXPLANATION=true` in `.env`, then restart the app. The first run downloads model files and can take t

Troubleshooting

- **GEMINI_API_KEY is missing:** confirm `.env` is in the project root and restart Uvicorn.
- **Invalid API key / quota error:** check the key, API access, and usage limits in Google AI Studio.
- **Module not found:** activate `.venv` and run `pip install -r requirements.txt`.
- **Port busy:** run `uvicorn main:app --reload --port 8001` and visit port 8001.
- **Quiz parse error:** retry with a shorter, clearer passage; model output is validated before display.

AI responses can be inaccurate. Verify important study information with textbooks or teachers. This project does not include user accounts or pers

qna.py

```
from ai_client import generate_text
```

```
async def answer_question(text: str, level: str = "beginner"):
    prompt = f"""You are EduGenie, a careful educational assistant.
    Answer the student's question at a {level} level. Be accurate, clear, and concise.
    If the question is ambiguous, state the assumption. Do not invent facts.
```

```
Student question:
{text}
```

```
Return a helpful answer with short paragraphs or bullets when useful."""
    return {"answer": await generate_text(prompt)}
```

requirements.txt

```
fastapi>=0.115,<1.0
uvicorn[standard]>=0.30,<1.0
jinja2>=3.1,<4.0
python-dotenv>=1.0,<2.0
google-genai>=1.0,<2.0
# Optional: install only if using the local LaMini explanation model
transformers>=4.44,<5.0
torch>=2.2
sentencepiece>=0.2
```

learning_path.py

```
from ai_client import generate_text

async def get_learning_recommendations(text: str, level: str = "beginner"):
    prompt = f"""Create a practical learning path for the topic: {text}
    Learner's current level: {level}
    Organize it from beginner to advanced. Include ordered topics, a suggested timeline,
    practice activities, and useful resource types (such as documentation, videos, articles,
    or books). Do not invent specific URLs. Use clear headings and concise steps."""
    return {"learning_path": await generate_text(prompt)}
```

summary_module.py

```
from ai_client import generate_text

async def summarize_text(text: str, level: str = "beginner"):
    prompt = f"""Summarize the following educational passage for a {level} student.
    Keep the main ideas and important facts. Use simple language and do not add information.
    Passage:
    {text}"""
    return {"summary": await generate_text(prompt)}
```

quiz_module.py

```
import json
import re
from ai_client import generate_text

def clean_json_block(value: str) -> str:
    value = value.strip()
    value = re.sub(r"^\s*(?:json)?\s*$", "", value, flags=re.I)
    value = re.sub(r"\s*```\s*$", "", value)
    start, end = value.find("[", value.rfind("]")
    if start >= 0 and end > start:
        return value[start:end + 1]
    start, end = value.find("{", value.rfind("}")
    if start >= 0 and end > start:
        return value[start:end + 1]
    return value

async def generate_quiz(text: str, level: str = "beginner"):
    prompt = f"""Create exactly 3 multiple-choice questions based only on the study text below.
    Student level: {level}
    Return ONLY a valid JSON array. Each item must have:
    "question": string,
    "options": an array of exactly 4 strings,
    "correct_answer": the exact text of one option,
    "explanation": a short explanation.
    Study text:
    {text}"""
    raw = await generate_text(prompt)
    try:
        data = json.loads(clean_json_block(raw))
        if isinstance(data, dict):
            data = data.get("questions", [])
        if not isinstance(data, list) or len(data) != 3:
            raise ValueError("Expected a JSON array containing exactly 3 questions.")
        for item in data:
            if not all(k in item for k in ("question", "options", "correct_answer")):
                raise ValueError("A question is missing required fields.")
            if len(item["options"]) != 4 or item["correct_answer"] not in item["options"]:
                raise ValueError("Each question must have 4 options and a matching correct answer.")
        return {"questions": data}
    except (json.JSONDecodeError, TypeError, ValueError) as exc:
        raise RuntimeError(f"Could not parse the quiz response: {exc}. Please try again.") from exc
```

.gitignore

```
.venv/
__pycache__/
*.py[cod]
.env
.cache/
.pytest_cache/
```

.env.example

```
# Get a Gemini API key from Google AI Studio. Keep this file private.
GEMINI_API_KEY=replace_with_your_api_key
GEMINI_MODEL=gemini-2.5-flash

# Set true to use the optional local LaMini model for /explain.
# First model download can be large and may require substantial RAM.
USE_LOCAL_EXPLANATION=false
LOCAL_EXPLANATION_MODEL=MBZUAI/LaMini-Flan-T5-783M
```

explanation_module.py

```
"""Concept explanation with an optional local LaMini model and Gemini fallback."""
import os
from functools import lru_cache
from ai_client import generate_text

@lru_cache(maxsize=1)
def _local_pipeline():
    from transformers import pipeline
    model_name = os.getenv("LOCAL_EXPLANATION_MODEL", "MBZUAI/LaMini-Flan-T5-783M")
    return pipeline("text2text-generation", model=model_name, device=-1)

async def explain_topic(text: str, level: str = "beginner"):
    use_local = os.getenv("USE_LOCAL_EXPLANATION", "false").lower() == "true"
    if use_local:
        import asyncio
        def run_local():
            pipe = _local_pipeline()
            prompt = f"Explain {text} in simple language for a {level} student. Give a definition and one easy example."
            result = pipe(prompt, max_new_tokens=220, do_sample=False)
            return result[0]["generated_text"].strip()
        try:
            answer = await asyncio.to_thread(run_local)
            return {"explanation": answer, "model": "local"}
        except Exception as exc:
            # A configured local model failure is surfaced rather than silently using cloud.
            raise RuntimeError(f"Local explanation model failed: {exc}") from exc
    prompt = f"""Explain the following topic to a {level} student using simple language.
    Include a short definition, 3 key points, and one easy example. Avoid unsupported claims.

    Topic: {text}"""
    return {"explanation": await generate_text(prompt), "model": "gemini"}
```

main.py

```
import os
from pathlib import Path
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel, Field
from dotenv import load_dotenv

load_dotenv(BASE_DIR / ".env")

from qna import answer_question
from explanation_module import explain_topic
from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import get_learning_recommendations

BASE_DIR = Path(__file__).resolve().parent
app = FastAPI(title="EduGenie - AI Learning Assistant", version="1.0.0")
app.mount("/static", StaticFiles(directory=BASE_DIR / "static"), name="static")
templates = Jinja2Templates(directory=BASE_DIR / "templates")

class TextRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=30000)
    level: str = "beginner"

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@app.get("/health")
async def health():
    return {"status": "ok"}

@app.post("/qa")
async def qa(payload: TextRequest):
    return await run_task(answer_question, payload)

@app.post("/explain")
async def explain(payload: TextRequest):
    return await run_task(explain_topic, payload)

@app.post("/quiz")
async def quiz(payload: TextRequest):
    return await run_task(generate_quiz, payload)

@app.post("/summarize")
async def summarize(payload: TextRequest):
    return await run_task(summarize_text, payload)

@app.post("/learn/recommendations")
async def recommendations(payload: TextRequest):
    return await run_task(get_learning_recommendations, payload)

async def run_task(fn, payload):
    try:
        result = await fn(payload.text, payload.level)
        return result if isinstance(result, dict) else {"result": result}
    except Exception as exc:
        return JSONResponse(status_code=500, content={"detail": str(exc)})
```

static/style.css

```
:root { font-family: Inter, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif; color: #1e293b; background: #f4f7fb; }
* { box-sizing: border-box; }
body { margin: 0; min-height: 100vh; }
.container { width: min(780px, calc(100% - 32px)); margin: 42px auto; }
header { display: flex; align-items: center; gap: 14px; margin-bottom: 24px; }
.brand-mark { width: 52px; height: 52px; display: grid; place-items: center; border-radius: 16px; background: #5b5ce2; color: white; font-size: 28px; }
h1 { margin: 0; font-size: 27px; } header p { margin: 3px 0 0; color: #64748b; }
.card { background: white; padding: 26px; border-radius: 18px; box-shadow: 0 8px 30px #1e293b0d; margin-bottom: 18px; }
h2 { margin: 0 0 20px; font-size: 20px; }
label { display: block; font-weight: 600; font-size: 14px; margin: 16px 0 7px; }
select, textarea { width: 100%; border: 1px solid #cbd5e1; border-radius: 10px; padding: 12px; font: inherit; background: white; color: #1e293b; }
textarea { resize: vertical; line-height: 1.5; }
select:focus, textarea:focus { outline: 2px solid #a5b4fc; border-color: #6366f1; }
button { border: 0; border-radius: 10px; padding: 12px 18px; margin-top: 18px; background: #5557d9; color: white; font: inherit; font-weight: 700; }
button:disabled { opacity: .6; cursor: wait; }
button.secondary { margin-top: 10px; padding: 9px 13px; background: #e0e7ff; color: #3730a3; }
.hint, footer { color: #64748b; font-size: 12px; } .hint { margin-bottom: 0; }
.result h2 { margin-bottom: 12px; }
.text-result { white-space: pre-wrap; line-height: 1.65; }
.question { border-top: 1px solid #e2e8f0; padding: 16px 0; }
.question h3 { font-size: 16px; margin: 0 0 10px; }
.choices label { font-weight: 400; margin: 8px 0; display: flex; gap: 7px; align-items: flex-start; }
.feedback { font-size: 14px; color: #334155; line-height: 1.5; }
footer { text-align: center; padding: 8px; }
@media (max-width: 520px) { .container { margin: 22px auto; } .card { padding: 20px; } }
```

templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EduGenie | Learning Assistant</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <main class="container">
    <header>
      <div class="brand-mark">E</div>
      <div><h1>EduGenie</h1><p>Your AI-powered learning companion</p></div>
    </header>
    <section class="card">
      <h2>What would you like to learn today?</h2>
      <label for="task">Choose a learning task</label>
      <select id="task">
        <option value="qa">Ask a question</option>
        <option value="explain">Explain a topic</option>
        <option value="quiz">Generate a quiz</option>
        <option value="summarize">Summarize text</option>
        <option value="learn/recommendations">Recommend a learning path</option>
      </select>
      <label for="level">Learning level</label>
      <select id="level">
        <option value="beginner">Beginner</option>
        <option value="intermediate">Intermediate</option>
        <option value="advanced">Advanced</option>
      </select>
      <label for="input">Your question, topic, or study text</label>
      <textarea id="input" rows="7" placeholder="For example: Explain photosynthesis in simple words..."></textarea>
      <button id="submit">Generate answer</button>
      <p class="hint">Avoid entering personal or sensitive information.</p>
    </section>
    <section id="result" class="card result" aria-live="polite" hidden>
      <h2>Result</h2><div id="output"></div>
    </section>
    <footer>EduGenie · Learning made simpler</footer>
  </main>
<script>
  const task = document.getElementById("task");
  const input = document.getElementById("input");
  const submit = document.getElementById("submit");
  const result = document.getElementById("result");
  const output = document.getElementById("output");

  task.addEventListener("change", () => {
    const prompts = {
      qa: "Ask a question...",
      explain: "Enter a topic to explain...",
      quiz: "Paste a passage or enter a topic...",
      summarize: "Paste the passage you want summarized...",
      "learn/recommendations": "Enter a topic for your learning path..."
    };
    input.placeholder = prompts[task.value];
  });

  function render(data) {
    output.replaceChildren();
    if (data.questions) {
      data.questions.forEach((q, i) => {
        const box = document.createElement("div");
        box.className = "question";
        const title = document.createElement("h3");
        title.textContent = `${i + 1}. ${q.question}`;
        box.appendChild(title);
        const choices = document.createElement("div");
        choices.className = "choices";
        q.options.forEach(option => {
          const label = document.createElement("label");
          const radio = document.createElement("input");
          radio.type = "radio"; radio.name = `q${i}`; radio.value = option;
          label.appendChild(radio, document.createTextNode(" " + option));
          choices.appendChild(label);
        });
        const check = document.createElement("button");
        check.className = "secondary";
        check.textContent = "Check answer";
        const feedback = document.createElement("p");
        feedback.className = "feedback";
        check.addEventListener("click", () => {
          const selected = box.querySelector(`input[name="q${i}"]:checked`);
          if (!selected) { feedback.textContent = "Choose an option first."; return; }
          feedback.textContent = selected.value === q.correct_answer
            ? "Correct! " + (q.explanation || "")
            : "Not quite. Correct answer: " + q.correct_answer + ". " + (q.explanation || "");
        });
        box.appendChild(choices, check, feedback);
        output.appendChild(box);
      });
    } else {
      const key = Object.keys(data).find(k => ["answer", "explanation", "summary", "learning_path", "result"].includes(k));
      const content = document.createElement("div");
      content.className = "text-result";
      content.textContent = key ? data[key] : JSON.stringify(data, null, 2);
    }
  }
</script>
```

```

        output.appendChild(content);
    }
}

submit.addEventListener("click", async () => {
    const text = input.value.trim();
    if (!text) { input.focus(); return; }
    submit.disabled = true; submit.textContent = "Working...";
    result.hidden = false; output.textContent = "Generating your response...";
    try {
        const response = await fetch("/" + task.value, {
            method: "POST",
            headers: {"Content-Type": "application/json"},
            body: JSON.stringify({text, level: document.getElementById("level").value})
        });
        const data = await response.json();
        if (!response.ok) throw new Error(data.detail || "Request failed.");
        render(data);
    } catch (error) {
        output.textContent = error.message || "Something went wrong.";
    } finally {
        submit.disabled = false; submit.textContent = "Generate answer";
    }
});
</script>
</body>
</html>

```